

Create a Spring Web Project using Maven

src > main > java > J LoggingExample.java > ...

```
1  import org.slf4j.Logger;
2  import org.slf4j.LoggerFactory;
3
4  public class LoggingExample {
5      private static final Logger logger = LoggerFactory.getLogger(LoggingExample.class);
6
7      public static void main(String[] args) {
8          logger.error("This is an error message");
9          logger.warn("This is a warning message");
10     }
11 }
12
```

pom.xml > () Grammars > https://maven.apache.org/xsd/maven-4.0.0.xsd > Cache > file:///C:/Users/prane/lemminx/cache/https://maven.apache.org/xsd/maven-4.0.0.xsd

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
4    <modelVersion>4.0.0</modelVersion>
5    <parent>
6      <groupId>org.springframework.boot</groupId>
7      <artifactId>spring-boot-starter-parent</artifactId>
8      <version>4.1.0</version>
9      <relativePath/> <!-- lookup parent from repository -->
10    </parent>
11    <groupId>com.cognizant</groupId>
12    <artifactId>spring-learn</artifactId>
13    <version>0.0.1-SNAPSHOT</version>
14    <name/>
15    <description/>
16    <url/>
17    <licenses>
18      <license/>
19    </licenses>
20    <developers>
21      <developer/>
22    </developers>
23    <scm>
24      <connection/>
25      <developerConnection/>
26      <tag/>
27      <url/>
28    </scm>
29    <properties>
30      <java.version>17</java.version>
31    </properties>
32    <dependencies> Add Spring Boot Starters...
33    <dependency>
34      <groupId>org.springframework.boot</groupId>
35      <artifactId>spring-boot-starter-webmvc</artifactId>
36    </dependency>
37
38    <dependency>
39      <groupId>org.springframework.boot</groupId>
40      <artifactId>spring-boot-devtools</artifactId>
41      <scope>runtime</scope>
42      <optional>true</optional>
43    </dependency>
44    <dependency>
45      <groupId>org.springframework.boot</groupId>
46      <artifactId>spring-boot-starter-webmvc-test</artifactId>
47      <scope>test</scope>
48    </dependency>
49    </dependencies>
50
51    <build>
52      <plugins>
53        <plugin>
54          <groupId>org.springframework.boot</groupId>
55          <artifactId>spring-boot-maven-plugin</artifactId>
56        </plugin>
57      </plugins>
58    </build>
59  </project>
```

: Graceful shutdown complete

```
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 01:28 min
[INFO] Finished at: 2026-07-08T18:52:35+05:30
[INFO] -----
```

Spring Core – Load Country from Spring Configuration XML

```
src > main > java > com > cognizant > springlearn > J Country.java > {} com.cognizant.springlearn
1 package com.cognizant.springlearn;
2
3 public class Country {
4
5     private String code;
6     private String name;
7
8     /**
9      * Default constructor
10     */
11     public Country() {
12         System.out.println(x: "Country() - Constructor invoked");
13     }
14
15     /**
16      * Parameterized constructor
17     */
18     public Country(String code, String name) {
19         this.code = code;
20         this.name = name;
21         System.out.println(x: "Country(String code, String name) - Constructor invoked");
22     }
23
24     /**
25      * Getter for code
26     */
27     public String getCode() {
28         return code;
29     }
30
31     /**
32      * Setter for code
33     */
34     public void setCode(String code) {
35         System.out.println("setCode() - Method invoked with value: " + code);
36         this.code = code;
37     }
38
39     /**
40      * Getter for name
41     */
42     public String getName() {
43         return name;
44     }
45
46     /**
47      * Setter for name
48     */
49     public void setName(String name) {
50         System.out.println("setName() - Method invoked with value: " + name);
51         this.name = name;
52     }
53
54     @Override
55     public String toString() {
56         return "Country{" +
57             "code-" + code + '\'' +
58             ", name-" + name + '\'' +
59             '}';
60     }
61 }
62
```

```
src > main > java > com > cognizant > springlearn > J SpringLearnApplication.java > {} com.cognizant.springlearn
1 package com.cognizant.springlearn;
2
3 import org.springframework.context.ApplicationContext;
4 import org.springframework.context.support.ClassPathXmlApplicationContext;
5 import org.slf4j.Logger;
6 import org.slf4j.LoggerFactory;
7
8 /**
9  * Spring Learn Application
10  * This application demonstrates loading country configuration from Spring XML configuration file
11  */
12 public class SpringLearnApplication {
13
14     private static final Logger LOGGER = LoggerFactory.getLogger(SpringLearnApplication.class);
15
16     /**
17      * Main method - Entry point of the application
18     */
19     @SuppressWarnings("unused")
20     public static void main(String[] args) {
21         LOGGER.debug("main() - Method invoked");
22
23         // Display all countries
24         LOGGER.debug("----- Displaying all countries -----");
25         displayCountry(countryBeanId: "countryUS");
26         displayCountry(countryBeanId: "countryDE");
27         displayCountry(countryBeanId: "countryIN");
28         displayCountry(countryBeanId: "countryJP");
29
30         LOGGER.debug("----- Application execution completed -----");
31     }
32
33     /**
34      * Method to read country bean from spring configuration file and display details
35      * @param countryBeanId - The bean ID of the country to retrieve
36     */
37     private static void displayCountry(String countryBeanId) {
38         LOGGER.debug("\ndisplayCountry() - Method invoked for bean: {}", countryBeanId);
39
40         // Create ApplicationContext by loading the country.xml configuration file
41         ApplicationContext context = new ClassPathXmlApplicationContext("country.xml");
42         LOGGER.debug("ClassPathXmlApplicationContext created and country.xml loaded");
43
44         // Get the country bean from the context
45         Country country = context.getBean(countryBeanId, Country.class);
46         LOGGER.debug("getBean() method invoked to retrieve bean with id: {}", countryBeanId);
47
48         // Display the country details
49         LOGGER.debug("Country : {}", country.toString());
50         System.out.println(country.toString());
51     }
52 }
```

```
src > main > resources > country.xml > {} Grammars > http://www.springframework.org/schema/beans
1 <?xml version="1.0" encoding="UTF-8" ?>
2 <beans xmlns="http://www.springframework.org/schema/beans"
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4     xsi:schemaLocation="http://www.springframework.org/schema/beans
5         http://www.springframework.org/schema/beans/spring-beans.xsd">
6
7     <!-- United States Country Bean -->
8     <bean id="countryUS" class="com.cognizant.springlearn.Country">
9         <property name="code" value="US" />
10        <property name="name" value="United States" />
11    </bean>
12
13    <!-- Germany Country Bean -->
14    <bean id="countryDE" class="com.cognizant.springlearn.Country">
15        <property name="code" value="DE" />
16        <property name="name" value="Germany" />
17    </bean>
18
19    <!-- India Country Bean -->
20    <bean id="countryIN" class="com.cognizant.springlearn.Country">
21        <property name="code" value="IN" />
22        <property name="name" value="India" />
23    </bean>
24
25    <!-- Japan Country Bean -->
26    <bean id="countryJP" class="com.cognizant.springlearn.Country">
27        <property name="code" value="JP" />
28        <property name="name" value="Japan" />
29    </bean>
30
31 </beans>
```

```
src > main > resources > logback.xml > xml
1 <?xml version="1.0" encoding="UTF-8" ?>
2 <configuration>
3     <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
4         <encoder>
5             <pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n</pattern>
6         </encoder>
7     </appender>
8
9     <root level="DEBUG">
10        <appender-ref ref="CONSOLE" />
11    </root>
12
13    <logger name="com.cognizant.springlearn" level="DEBUG" />
14 </configuration>
15
```

```
PS D:\PROJECT\Spring> mvn exec:java
2026-07-09 15:21:55.223 [com.cognizant.springlearn.SpringLearnApplication.main()] DEBUG c.c.s.SpringLearnApplication - ===== Application execut
ion completed =====
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 2.771 s
[INFO] Finished at: 2026-07-09T15:21:55+05:30
[INFO] -----
```

Hello World RESTful Web Service

```
src > main > java > com > cognizant > springlearn > controller > J HelloController.java > Language Support for Java(TM) by Red Hat > { } com.cognizant:
1 package com.cognizant.springlearn.controller;
2
3 import org.slf4j.Logger;
4 import org.slf4j.LoggerFactory;
5 import org.springframework.web.bind.annotation.GetMapping;
6 import org.springframework.web.bind.annotation.RestController;
7
8 @RestController
9 public class HelloController {
10     private static final Logger logger = LoggerFactory.getLogger(HelloController.class);
11
12     @GetMapping("/hello")
13     public String sayHello() {
14         logger.info("Start sayHello()");
15         logger.info("End sayHello()");
16         return "Hello World!!";
17     }
18 }
19

src > main > java > com > cognizant > springlearn > J SpringLearnApplication.java > Language Support for Java(TM) by Red Hat > { } c
1 package com.cognizant.springlearn;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6 @SpringBootApplication
7 public class SpringLearnApplication {
8     Run | Debug
9     public static void main(String[] args) {
10         SpringApplication.run(SpringLearnApplication.class, args);
11     }
12 }
```

1 Hello World!!

Key	Value
Content-Type	text/plain;charset=UTF-8
Content-Length	13
Date	Thu, 09 Jul 2026 10:06:11 GMT
Keep-Alive	timeout=60
Connection	keep-alive

REST - Country Web Service

```
src > test > java > com > cognizant > springlearn > J CountryControllerIntegrationTest.java > {} com.cognizant.springlearn
1 package com.cognizant.springlearn;
2
3 import org.junit.jupiter.api.Test;
4 import org.springframework.beans.factory.annotation.Autowired;
5 import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
6 import org.springframework.boot.test.context.SpringBootTest;
7 import org.springframework.http.MediaType;
8 import org.springframework.test.web.servlet.MockMvc;
9
10 import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
11 import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.content;
12 import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
13 import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;
14
15 @SpringBootTest
16 @AutoConfigureMockMvc
17 class CountryControllerIntegrationTest {
18
19     @Autowired
20     private MockMvc mockMvc;
21
22     @Test
23     void getCountryIndiaReturnsIndiaDetails() throws Exception {
24         mockMvc.perform(get("/country"))
25             .andExpect(status().isOk())
26             .andExpect(content().contentTypeCompatibleWith(MediaType.APPLICATION_JSON))
27             .andExpect(jsonPath("$.code").value("IN"))
28             .andExpect(jsonPath("$.name").value("India"));
29     }
30 }
31
```

```
src > main > java > com > cognizant > springlearn > J CountryWebServiceApplication.java > Language Support for Java(TM) by Red Hat > {} com.cognizant.springlearn
1 package com.cognizant.springlearn;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5 import org.springframework.context.annotation.ImportResource;
6
7 @SpringBootApplication
8 @ImportResource("classpath:country-beans.xml")
9 public class CountryWebServiceApplication {
10     Run | Debug
11     public static void main(String[] args) {
12         SpringApplication.run(CountryWebServiceApplication.class, args);
13     }
14 }

```

```
1 {
2     "code": "IN",
3     "name": "India"
4 }
```

Key	Value
Content-Type	application/json
Transfer-Encoding	chunked
Date	Thu, 09 Jul 2026 10:46:03 GMT
Keep-Alive	timeout=60
Connection	keep-alive

REST - Get country based on country code

```
src > main > java > com > cognizant > springlearn > controller > J CountryController.java > Language Support for Java(TM) by Red Hat > {} com.cognizant.springlearn.controller

1 package com.cognizant.springlearn.controller;
2
3 import com.cognizant.springlearn.model.Country;
4 import com.cognizant.springlearn.service.CountryService;
5 import org.springframework.web.bind.annotation.GetMapping;
6 import org.springframework.web.bind.annotation.PathVariable;
7 import org.springframework.web.bind.annotation.RestController;
8
9 @RestController
10 public class CountryController {
11
12     private final CountryService countryService;
13
14     public CountryController(CountryService countryService) {
15         this.countryService = countryService;
16     }
17
18     @GetMapping("/countries/{code}")
19     public Country getCountry(@PathVariable String code) {
20         return countryService.getCountry(code);
21     }
22 }
23
```

```
src > main > java > com > cognizant > springlearn > model > J Country.java > Country

1 package com.cognizant.springlearn.model;
2
3 public class Country {
4     private String code;
5     private String name;
6
7     public Country() {
8     }
9
10    public Country(String code, String name) {
11        this.code = code;
12        this.name = name;
13    }
14
15    public String getCode() {
16        return code;
17    }
18
19    public void setCode(String code) {
20        this.code = code;
21    }
22
23    public String getName() {
24        return name;
25    }
26
27    public void setName(String name) {
28        this.name = name;
29    }
30 }
31
```

```
src > main > java > com > cognizant > springlearn > CountryCodeWebServiceApplication.java > Language Support for Java(TM) by Red H
1 package com.cognizant.springlearn;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6 @SpringBootApplication
7 public class CountryCodeWebServiceApplication {
8     Run | Debug
9     public static void main(String[] args) {
10         SpringApplication.run(CountryCodeWebServiceApplication.class, args);
11     }
12 }
```

```
1 {
2     "code": "IN",
3     "name": "India"
4 }
```

Key	Value
Content-Type	application/json
Transfer-Encoding	chunked
Date	Thu, 09 Jul 2026 11:02:08 GMT
Keep-Alive	timeout=60
Connection	keep-alive

Create authentication service that returns JWT

```
src > main > java > com > example > jwtservice > JwtServiceApplication.java > Language Support for Java(TM) by Red Hat > { } com.example.jwtservice
1 package com.example.jwtservice;
2
3
4 import org.springframework.boot.SpringApplication;
5 import org.springframework.boot.autoconfigure.SpringBootApplication;
6
7 @SpringBootApplication
8 public class JwtServiceApplication {
9     public static void main(String[] args) {
10         SpringApplication.run(JwtServiceApplication.class, args);
11     }
12 }
```

```
src > test > java > com > example > J AuthenticationControllerTest.java > {} com.example.jwtservice
1 package com.example.jwtservice;
2
3 import org.junit.jupiter.api.Test;
4 import org.springframework.beans.factory.annotation.Autowired;
5 import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
6 import org.springframework.boot.test.context.SpringBootTest;
7 import org.springframework.http.MediaType;
8 import org.springframework.test.web.servlet.MockMvc;
9
10 import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
11 import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
12 import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;
13
14 @SpringBootTest
15 @AutoConfigureMockMvc
16 class AuthenticationControllerTest {
17
18     @Autowired
19     private MockMvc mockMvc;
20
21     @Test
22     void shouldReturnTokenForBasicAuthCredentials() throws Exception {
23         mockMvc.perform(get("/authenticate")
24             .header("Authorization", "Basic dXNlcjpwd2Q=")
25             .accept(MediaType.APPLICATION_JSON)
26             .andExpect(status().isOk())
27             .andExpect(jsonPath("$.token").isString());
28     }
29 }
```

```
1 {  
2 |   "token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJlczVvYyIiwiaWF0IjoxNzg5NTk1OTk0LmEHAiojE3ODM1OTk1OTk0.61PuNFpO4bvbt0uUuxr3ryP1IhWT1AfMnTSQmEBZTYs"  
3 }
```